INSTITUTO ARGENTINO DE LA EMPRESA

Carrera: Analista de Sistemas de Información

Asignatura: Algoritmos II · Ciclo Lectivo 2026



TRABAJO PRÁCTICO FINAL

Sistema de Gestión de Coworking y Reservas

DOCUMENTO TÉCNICO Y MEMORIA INTEGRADORA

Arquitectura POO, Persistencia MySQL, Manejo de Errores y Manual de Operación

---

Integrantes del Equipo: Tullo, Jonathan Lester (shonimdp@gmail.com)
Ortega, David Samuel (davidortegaa86@gmail.com)
Scollo, Maximiliano (maximilianoscollo17@gmail.com)

Comisión / Curso: 2º Año · Modalidad Regular

Docente a cargo: Lic. Adrián Aroca

Ciclo lectivo: 2026

Fecha de entrega: Noviembre de 2026

Versión del documento: Versión 1.0 (Entrega Integrada Final)

ÍNDICE DE CONTENIDOS

1. Introducción y Objetivo General del Sistema **3**
2. Tecnologías, Herramientas y Lenguajes Utilizados **4**
3. Organización y Distribución del Trabajo en Equipo **5**
4. Modelo de Datos y Persistencia en MySQL **6**
4.1. Esquema Relacional y Restricciones de Integridad 6
4.2. Diagramas Entidad-Relación (DER Workbench y Conceptual) 6
4.3. Diccionario de Datos: Estructura de Tablas 7
4.4. Vistas del Sistema y Funciones Almacenadas 9
4.5. Procedimientos Almacenados y Reglas de Negocio con SIGNAL 45000 9
4.6. Trigger de Auditoría Automática 10
5. Arquitectura del Software y Programación Orientada a Objetos **11**
5.1. Arquitectura en Capas del Sistema 11
5.2. Diagrama de Clases Simplificado (UML) 11
5.3. Interfaz de Dominio (IReservable.cs) 12
5.4. Enumerador Tipado (EstadoReserva.cs) 12
5.5. Clase Abstracta Base (Recurso.cs) y Justificación de Diseño 12
5.6. Clases Derivadas Especializadas (Sala, OficinaPrivada, PuestoTrabajo) 13
5.7. Justificación del Polimorfismo: CalcularCosto(horas) 13
5.8. Clases de Entidad (Cliente.cs y Reserva.cs) 14
5.9. Clase Gestora en Memoria (GestoraCoworking.cs) y Consultas LINQ 14
5.10. Evolución hacia el Patrón DAO (GestorDAO.cs) 15
6. Manejo de Errores, Estrategia de Conexión y Decisiones de Diseño **16**
6.1. Estrategia de Conexión Centralizada (Conexion.cs) y App.config 16
6.2. Flujo de Captura del Error 45000 de MySQL a través de MySqlException 16
6.3. Manejo General de Excepciones y Ausencia de Bloques Vacíos 17
6.4. Funcionalidad y Esquema de Intercambio de Datos JSON 17
6.5. Justificación de Decisiones Técnicas y Alternativas Analizadas 18
7. Manual de Usuario y Guía de Operación del Sistema **19**
7.1. Requisitos Previos para el Encendido del Sistema 19
7.2. Autenticación de Usuarios (FrmLogin) y Asistencia vía WhatsApp 19
7.3. Pantalla Principal (FrmPrincipal): Control Operativo y Reservas 19
7.4. Módulo de Administración: Gestión de Usuarios y Recursos 20
8. Guía de Ejecución y Puesta en Marcha **21**
9. Referencias Bibliográficas (Normas APA 7.ª edición) **22**
10. Checklist Institucional de Control Previo a la Entrega **23**

1. Introducción y Objetivo General del Sistema

El presente proyecto corresponde al Trabajo Práctico Final integrador de la asignatura Algoritmos II, perteneciente a la carrera de Analista de Sistemas de Información del Instituto Argentino de la Empresa. Consiste en el diseño, desarrollo e implementación de una aplicación de escritorio completa construida en el lenguaje C# bajo la plataforma .NET Framework 4.7.2 con interfaz gráfica en Windows Forms, conectada a un motor de base de datos relacional MySQL/MariaDB bajo el entorno XAMPP.

El propósito del sistema es administrar de forma integral las actividades operativas y administrativas de un espacio de trabajo colaborativo (Coworking). Permite gestionar el padrón de clientes, catalogar los espacios físicos disponibles para alquiler (salas de reuniones, oficinas privadas y puestos de trabajo individuales) y registrar reservas horarias con control riguroso de concurrencia y disponibilidad.

La solución resuelve integralmente las reglas críticas del negocio: impide la superposición o solapamiento horario de reservas sobre un mismo recurso mediante lógica matemática encapsulada en procedimientos y funciones almacenadas de MySQL (emitiendo excepciones controladas vía SIGNAL SQLSTATE '45000'), calcula dinámicamente las tarifas aplicando polimorfismo orientado a objetos según las comodidades específicas de cada espacio, y audita automáticamente toda cancelación de servicio a través de un disparador (trigger) en la base de datos.

2. Tecnologías, Herramientas y Lenguajes Utilizados

Para la construcción de la solución se adoptaron tecnologías estandarizadas, garantizando robustez, mantenibilidad y rendimiento óptimo:

- **C# (.NET Framework 4.7.2):** Lenguaje principal de desarrollo con tipado estático riguroso, programación orientada a objetos (clases abstractas, interfaces, herencia y polimorfismo), colecciones genéricas y lenguaje de consulta integrado (LINQ).
- **Windows Forms (WinForms):** Framework para la construcción de la interfaz gráfica de usuario en entornos Windows, empleando formularios desacoplados, grillas de datos (DataGridView), selectores de fecha gráficos (DateTimePicker), controles de pestañas (TabControl) y controles numéricos (NumericUpDown).
- **MySQL / MariaDB (v10.4+):** Sistema gestor de bases de datos relacionales alojado en localhost:3306 bajo XAMPP, responsable de la persistencia de datos, integridad referencial (claves primarias y foráneas), vistas analíticas, funciones determinísticas, disparadores de auditoría y procedimientos almacenados.
- **MySql.Data:** Conector oficial de Oracle para ADO.NET (v9.5.0), utilizado para la administración de conexiones (MySqlConnection), ejecución de comandos parametrizados (MySqlCommand), manejo de parámetros de entrada/salida y captura de excepciones específicas (MySqlException).
- **Microsoft Visual Studio 2022:** Entorno de desarrollo integrado utilizado para la escritura de código, diseño de formularios, depuración paso a paso, compilación mediante MSBuild y gestión de paquetes vía NuGet.
- **MySQL Workbench 8.0 CE y phpMyAdmin:** Herramientas oficiales de diseño relacional y administración de base de datos para la diagramación del modelo (mwb) y la ejecución del script de despliegue (DB_TPFinal.sql).
- **Git y GitHub:** Sistema de control de versiones distribuido y repositorio remoto para la sincronización colaborativa y trazabilidad histórica de cambios del equipo.
- **Protocolo Web WhatsApp:** Mecanismo de enlace URI (wa.me) para derivar consultas operativas directamente a la línea de asistencia técnica del coworking.

3. Organización y Distribución del Trabajo en Equipo

El desarrollo se articuló en módulos funcionales asignados a cada integrante del equipo, promoviendo la especialización técnica y la integración continua:

- **Jonathan Lester Tullo (shonimdp@gmail.com):** Diseño relacional y programación del script completo de base de datos MySQL (DB_TPFinal.sql): tablas, restricciones, vistas, funciones, procedimientos almacenados y trigger de auditoría. Configuración del proyecto base en Visual Studio 2022 (.NET 4.7.2), resolución de dependencias NuGet (MySql.Data), programación de la clase centralizada de conexión (Conexion.cs) y desarrollo del modelo base (Recurso.cs, IReservable.cs, EstadoReserva.cs).
- **Maximiliano Scollo (maximilianoscollo17@gmail.com):** Programación de las clases derivadas del modelo (Sala.cs, OficinaPrivada.cs, PuestoTrabajo.cs) con implementación del cálculo polimórfico de costos (CalcularCosto) y disponibilidad (PuedeReservarse). Desarrollo de los modelos Cliente.cs y Reserva.cs, implementación de la clase gestora en memoria con LINQ (GestoraCoworking.cs) y diagramación del modelo relacional en MySQL Workbench.
- **David Samuel Ortega (davidortegaa86@gmail.com):** Diseño e implementación de la interfaz gráfica en Windows Forms: desarrollo de la pantalla de autenticación (FrmLogin) con control de credenciales y asistencia técnica vía WhatsApp, preparación de recursos gráficos y maquetado de la pantalla principal (FrmPrincipal) con solapas operativas (Control y Reservas) y solapa administrativa protegida por roles (Gestión).

4. Modelo de Datos y Persistencia en MySQL

La persistencia del sistema está gobernada por la base de datos tp_final_db alojada en MySQL. El diseño respeta rigurosamente la Tercera Forma Normal (3NF), garantizando la no duplicación de información y la preservación de la integridad referencial.

4.1. Esquema Relacional y Restricciones de Integridad

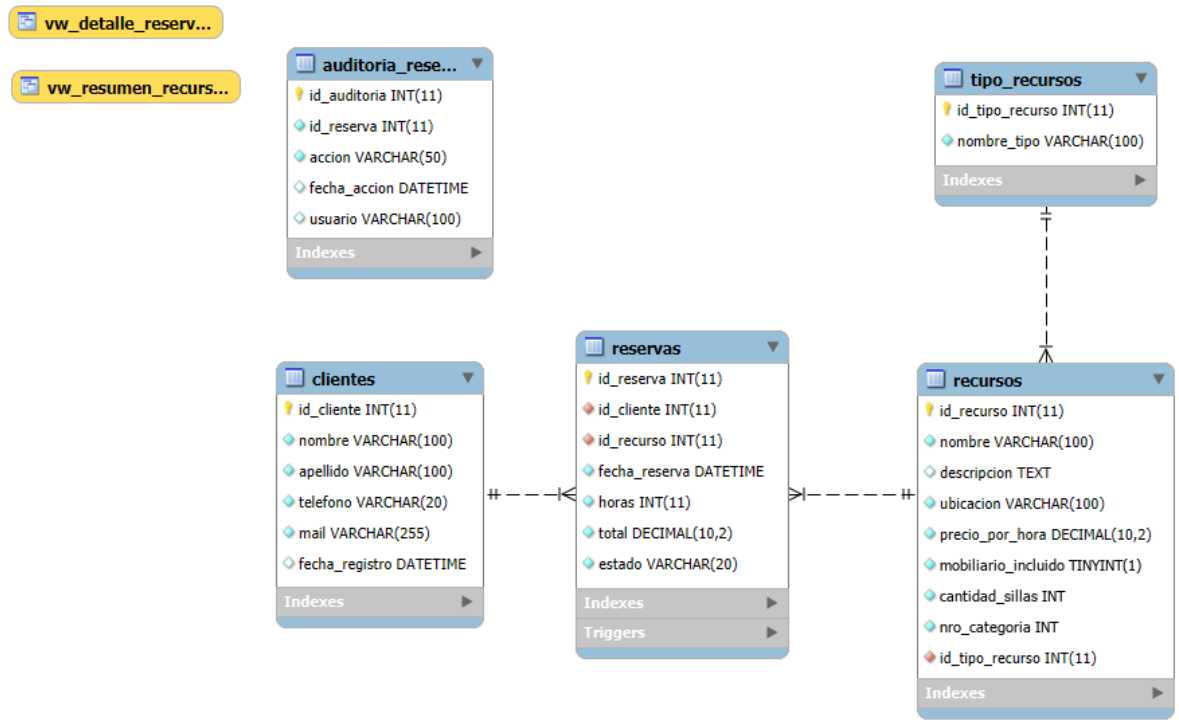
El modelo relacional vincula clientes, recursos físicos y sus reservas temporales. Se aplican claves foráneas con restricciones estrictas de integridad: no es posible generar reservas asociadas a clientes inexistentes ni a recursos que no figuren en el catálogo general.

4.2. Diagramas Entidad-Relación (DER Workbench y Conceptual)

A continuación se presentan el diagrama exportado de MySQL Workbench y el diagrama conceptual de entidades:

Figura 1

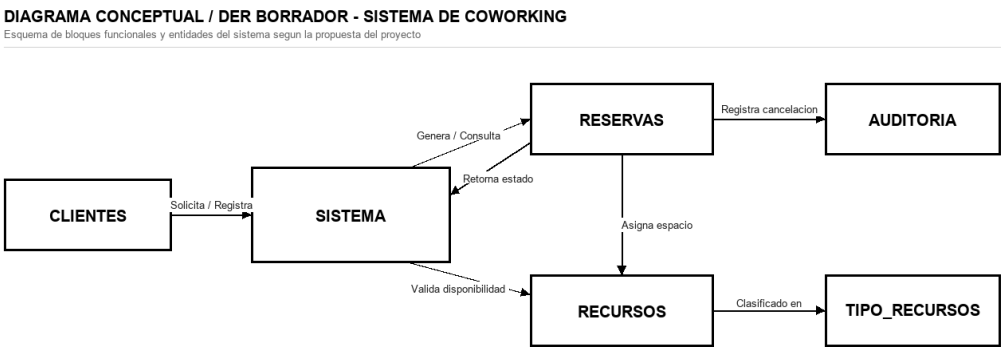
Diagrama Entidad-Relación (DER) del Modelo Relacional en MySQL Workbench



Nota. Exportación directa del modelo de datos de la base tp_final_db generada en MySQL Workbench 8.0 CE.

Figura 2

Diagrama Conceptual de Bloques y Flujo de Entidades del Negocio



Nota. Esquema conceptual inicial consensuado por el equipo de desarrollo para modelar las interacciones de coworking.

4.3. Diccionario de Datos: Estructura de Tablas

Tabla: tipo_recursos (Categorización de espacios)

Campo	Tipo de Dato	Nulo	Clave	Descripción
id_tipo_recurso	INT	NO	PK (AI)	Identificador unívoco del tipo de espacio físico.
nombre_tipo	VARCHAR(100)	NO	-	Denominación de la categoría (Sala, Oficina, Puesto).

Tabla: recursos (Espacios físicos comercializables)

Campo	Tipo de Dato	Nulo	Clave	Descripción
id_recurso	INT	NO	PK (AI)	Identificador unívoco del recurso físico.
nombre	VARCHAR(100)	NO	-	Nombre comercial del espacio (ej. Sala Innovación).
descripcion	TEXT	SÍ	-	Descripción de comodidades y equipamiento disponible.
ubicacion	VARCHAR(100)	NO	-	Sector y nivel edilicio (ej. Piso 2 - Sector B).
precio_por_hora	DECIMAL(10,2)	NO	-	Tarifa monetaria base por hora de uso.
mobiliario_incluido	BOOLEAN	SÍ	-	Aplica a oficinas: indica si incluye mobiliario completo.

Campo	Tipo de Dato	Nulo	Clave	Descripción
cantidad_sillas	INT	SÍ	-	Aplica a salas: capacidad total de sillas instaladas.
nro_categoria	INT	SÍ	-	Aplica a puestos: nivel del box de trabajo (categorías 1, 2 o 3).
id_tipo_recurso	INT	NO	FK	Referencia foránea a tipo_recursos.id_tipo_recurso.

Tabla: clientes (Padrón de usuarios del coworking)

Campo	Tipo de Dato	Nulo	Clave	Descripción
id_cliente	INT	NO	PK (AI)	Identificador unívoco del cliente.
nombre	VARCHAR(100)	NO	-	Nombre de pila del cliente.
apellido	VARCHAR(100)	NO	-	Apellido del cliente.
telefono	VARCHAR(20)	NO	-	Teléfono de contacto directo.
mail	VARCHAR(255)	NO	-	Dirección de correo electrónico institucional o personal.
fecha_registro	DATETIME	NO	Default Now	Estampa temporal de alta del cliente en el sistema.

Tabla: reservas (Tabla relacional pivote de transacciones)

Campo	Tipo de Dato	Nulo	Clave	Descripción
id_reserva	INT	NO	PK (AI)	Identificador unívoco de la reserva.
id_cliente	INT	NO	FK	Referencia foránea al cliente titular (clientes.id_cliente).
id_recurso	INT	NO	FK	Referencia foránea al espacio asignado (recursos.id_recurso).
fecha_reserva	DATETIME	NO	-	Fecha y hora pactada para el inicio del alquiler.
horas	INT	NO	Default 1	Duración en horas del período de uso contratado.
total	DECIMAL(10,2)	NO	-	Importe total facturado calculado para la operación.
estado	VARCHAR(20)	NO	Default Conf	Estado operativo: 'Confirmada', 'Cancelada' o 'Pendiente'.

Tabla: auditoria_reservas (Trazabilidad y registro de cancelaciones)

Campo	Tipo de Dato	Nulo	Clave	Descripción

Campo	Tipo de Dato	Nulo	Clave	Descripción
id_auditoria	INT	NO	PK (AI)	Identificador secuencial del evento de auditoría.
id_reserva	INT	NO	-	Número de la reserva afectada por la cancelación.
accion	VARCHAR(50)	NO	-	Descripción del suceso ('Cancelacion de reserva').
fecha_accion	DATETIME	NO	Default Now	Fecha y hora exacta en la que se produjo la baja.
usuario	VARCHAR(100)	SÍ	Default Sist	Identificador de usuario o sistema que ejecutó la baja.

4.4. Vistas del Sistema y Funciones Almacenadas

- **vw_detalle_reservas:** Combina mediante INNER JOIN las tablas reservas, clientes y recursos, proyectando nombres, apellidos, denominación del espacio físico, fecha, duración y estado en texto claro, desacoplando a la capa de presentación de la complejidad relacional de IDs.
- **vw_resumen_recursos:** Agrupa por recurso mediante GROUP BY calculando la cantidad acumulada de reservas confirmadas y el monto total recaudado, proporcionando resúmenes analíticos directos.
- **fn_verificar_disponibilidad:** Recibe el identificador del recurso, la fecha/hora de inicio y la duración requerida. Evalúa la superposición de intervalos temporales aplicando la fórmula matemática estricta: (nuevo_inicio < fin_existente) AND (nuevo_fin > inicio_existente) frente a reservas confirmadas. Retorna 1 si el espacio se encuentra disponible o 0 en caso de conflicto.

4.5. Procedimientos Almacenados y Reglas de Negocio con SIGNAL 45000

Las operaciones de modificación de estado en la base de datos se ejecutan exclusivamente a través de Stored Procedures. En particular, sp_alta_reserva implementa una barrera de integridad de negocio inquebrantable:

```
-- Procedimiento Almacenado con validación de disponibilidad y SIGNAL 45000
DELIMITER $$
CREATE PROCEDURE sp_alta_reserva(
    IN p_id_cliente INT,
    IN p_id_recurso INT,
    IN p_fecha DATETIME,
    IN p_horas INT,
    IN p_total DECIMAL(10,2),
    IN p_estado VARCHAR(20)
)
BEGIN
    -- Validación de horas e importe
```

```
    IF p_horas <= 0 THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'La cantidad de horas debe ser mayor a cero.';
    END IF;

    -- Validación matemática de superposición de intervalos
    IF fn_verificar_disponibilidad(p_id_recurso, p_fecha, p_horas) = 0 THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'El recurso ya se encuentra reservado u ocupado en ese
intervalo de horario.';
    END IF;

    -- Inserción si pasa todas las reglas de negocio
    INSERT INTO reservas (id_cliente, id_recurso, fecha_reserva, horas, total, estado)
    VALUES (p_id_cliente, p_id_recurso, p_fecha, p_horas, p_total, IFNULL(p_estado,
'Confirmada'));
END$$
DELIMITER ;
```

Código 1: sp_alta_reserva con validación de solapamiento y emisión de SIGNAL SQLSTATE '45000'

4.6. Trigger de Auditoría Automática

El trigger trg_auditoria_reservas opera de manera reactiva tras una actualización (AFTER UPDATE) en la tabla reservas. Examina si el estado mutó a 'Cancelada' habiendo estado previamente en otro estado. De cumplirse, inserta automáticamente un registro histórico en auditoria_reservas, garantizando trazabilidad inmutable sin depender del código de la interfaz.

5. Arquitectura del Software y Programación Orientada a Objetos

El desarrollo del software en C# adopta los principios fundamentales de la Programación Orientada a Objetos (POO): abstracción, encapsulamiento, herencia funcional y polimorfismo dinámico.

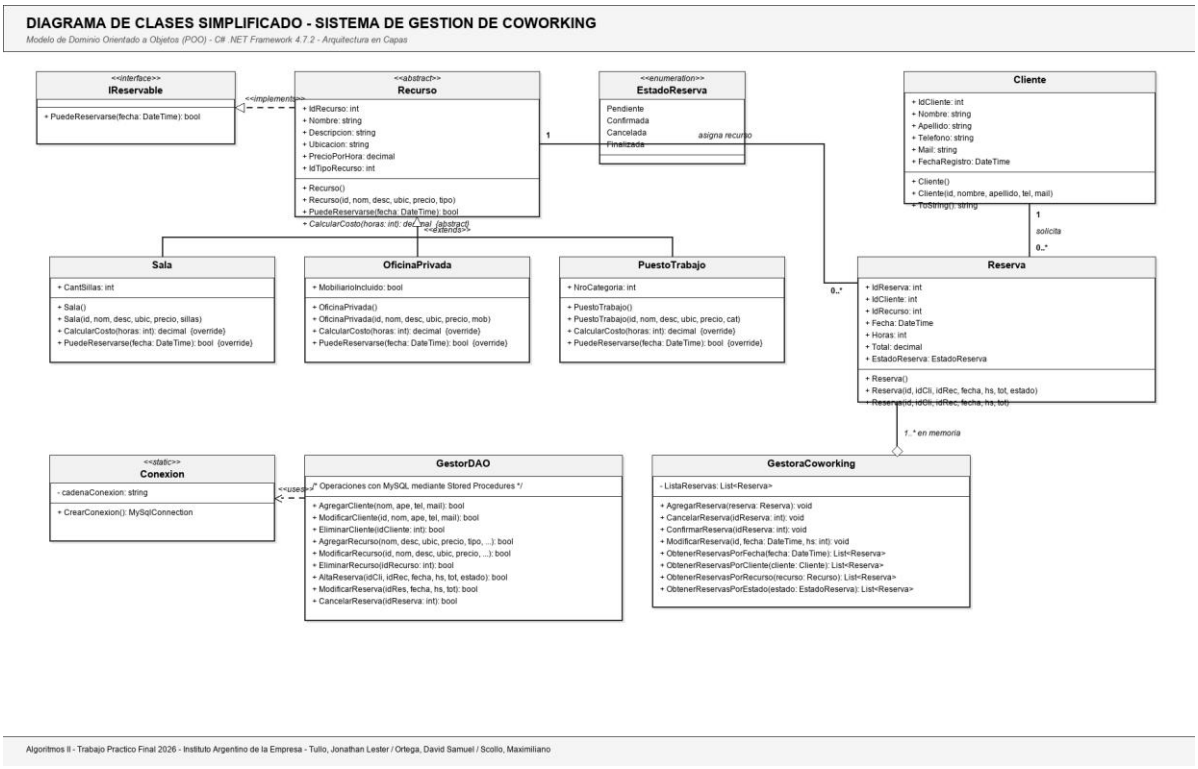
5.1. Arquitectura en Capas del Sistema

- **Capa de Presentación (UI):** Formularios FrmLogin y FrmPrincipal. Capturan interacciones de usuario, configuran controles y muestran respuestas visuales sin contener lógica relacional.
- **Capa de Negocio (BLL):** Clases gestoras (GestoraCoworking) que procesan listas en memoria, aplican filtros LINQ y coordinan reglas de negocio.
- **Capa de Modelos (Entidades):** Clases de dominio (Recurso, Sala, OficinaPrivada, PuestoTrabajo, Cliente, Reserva) que modelan las entidades y encapsulan sus atributos y cálculos.
- **Capa de Acceso a Datos (DAO):** Clase GestorDAO encargada de aislar el consumo de procedimientos almacenados, comandos y parámetros ADO.NET.
- **Capa de Infraestructura / Conexión:** Clase estática Conexion responsable de fabricar instancias MySqlConnection hacia el servidor MySQL.

5.2. Diagrama de Clases Simplificado (UML)

Figura 3

Diagrama de Clases Simplificado del Dominio (UML)



Nota. Diagrama UML con especificación de atributos, métodos, jerarquía de herencia (Recurso) e implementación de interfaz (IReservable).

5.3. Interfaz de Dominio (IReservable.cs)

La interfaz IReservable define el contrato formal bool PuedeReservarse(DateTime fecha). Su utilización desacopla la capacidad de reserva de la jerarquía de herencia de clases: cualquier recurso físico, servicio adicional o equipamiento tecnológico que deba validar disponibilidad en una fecha determinada implementa esta interfaz de forma uniforme.

5.4. Enumerador Tipado (EstadoReserva.cs)

El enumerador EstadoReserva (con valores Pendiente, Confirmada, Cancelada, Finalizada) elimina la utilización de cadenas literales arbitrarias en el código, mitigando errores de tipeo y dotando a la lógica de negocio de fuerte tipado estático en tiempo de compilación.

5.5. Clase Abstracta Base (Recurso.cs) y Justificación de Diseño

La clase Recurso se define con el modificador abstract porque en el mundo real no existe un 'recurso genérico' que un cliente pueda alquilar; los clientes alquilan espacios concretos

con características físicas determinadas (una sala de conferencias con cierta cantidad de sillas, una oficina amueblada o un puesto individual de categoría específica). Recurso reúne los atributos comunes (IdRecurso, Nombre, Descripcion, Ubicacion, PrecioPorHora, IdTipoRecurso), implementa la interfaz IReservable y establece la firma abstracta obligatoria public abstract decimal CalcularCosto(int horas); forzando a que cada subclase provea su propio algoritmo de cálculo.

5.6. Clases Derivadas Especializadas

Las subclases especializan el comportamiento base incorporando atributos diferenciales del negocio:

- **Sala (IdTipoRecurso = 1):** Añade la propiedad CantSillas. En el cálculo de costo, evalúa la capacidad de asistentes para incorporar cargos de mantenimiento y limpieza.
- **OficinaPrivada (IdTipoRecurso = 2):** Añade la propiedad booleana MobiliarioIncluido. Si la oficina cuenta con equipamiento ergonómico y mobiliario ejecutivo completo, incorpora un canon fijo adicional.
- **PuestoTrabajo (IdTipoRecurso = 3):** Añade la propiedad numérica NroCategoria (1, 2 o 3). Aplica un plus tarifario escalonado según el nivel de comodidades tecnológicas del box.

5.7. Justificación del Polimorfismo: CalcularCosto(horas)

El polimorfismo constituye el pilar arquitectónico central del subsistema de tarifación. Al definir CalcularCosto(int horas) como método abstracto en la clase base Recurso y sobrescribirlo mediante override en cada clase hija, el sistema permite que cualquier componente (como el formulario de reservas o la clase gestora) invoque el método sobre una referencia genérica Recurso sin necesidad de inspeccionar el tipo concreto en tiempo de ejecución.

Esta decisión técnica aporta dos ventajas críticas: elimina por completo el uso de estructuras condicionales acopladas (switch o if-else encadenados basados en tipos) y cumple de manera estricta con el Principio Abierto/Cerrado (OCP - Open/Closed Principle) de SOLID. Si en el futuro se incorporara una nueva clase de recurso (por ejemplo, CabinaDeStreaming o TerrazaDeEventos), bastará con crear una nueva clase derivada de Recurso que sobreescriba CalcularCosto, sin tener que alterar ni una sola línea del código de cálculo existente.

```
// Implementación de polimorfismo dinámico en las clases del dominio
public abstract class Recurso : IReservable
{
    public decimal PrecioPorHora { get; set; }
    // Método abstracto: obliga a la implementación en cada derivada
    public abstract decimal CalcularCosto(int horas);
```

```
}

public class Sala : Recurso
{
    public int CantSillas { get; set; }
    public override decimal CalcularCosto(int horas)
    {
        decimal costo = PrecioPorHora * horas;
        if (CantSillas > 60) costo += 16000;
        else if (CantSillas > 30) costo += 8000;
        return costo;
    }
}

public class OficinaPrivada : Recurso
{
    public bool MobiliarioIncluido { get; set; }
    public override decimal CalcularCosto(int horas)
    {
        decimal costo = PrecioPorHora * horas;
        return MobiliarioIncluido ? costo + 1500 : costo;
    }
}

public class PuestoTrabajo : Recurso
{
    public int NroCategoria { get; set; }
    public override decimal CalcularCosto(int horas)
    {
        decimal costo = PrecioPorHora * horas;
        if (NroCategoria == 1) return costo + 3500;
        if (NroCategoria == 2) return costo + 5500;
        if (NroCategoria == 3) return costo + 8000;
        return costo;
    }
}
```

Código 2: Sobrescritura polimórfica del método CalcularCosto(horas) en clases derivadas

5.8. Clases de Entidad (Cliente.cs y Reserva.cs)

Cliente modela los datos de filiación del titular (IdCliente, Nombre, Apellido, Telefono, Mail, FechaRegistro).

Reserva actúa como entidad relacional pura, vinculando al cliente con el espacio rentado, la estampa temporal de inicio, la duración en horas, el monto total facturado y el estado operativo tipado mediante EstadoReserva.

5.9. Clase Gestora en Memoria (GestoraCoworking.cs) y Consultas LINQ

GestoraCoworking administra una colección genérica List<Reserva> en memoria principal. Aplica el encapsulamiento de datos y expone métodos de consulta basados en expresiones lambda y Language Integrated Query (LINQ):

- **Búsqueda por fecha:** ListaReservas.Where(r => r.Fecha.Date == fecha.Date).ToList()
- **Filtrado por cliente:** ListaReservas.Where(r => r.IdCliente == cliente.IdCliente).ToList()

- **Filtrado por estado:** ListaReservas.Where(r => r.EstadoReserva == estado).ToList()
- **Localización unívoca:** ListaReservas.FirstOrDefault(r => r.IdReserva == idReserva)

5.10. Evolución hacia el Patrón DAO (GestorDAO.cs)

Para culminar el desacoplamiento entre la capa visual y el motor relacional conforme a las consignas evolutivas de la cátedra, el sistema incorpora GestorDAO.cs. Esta clase asume la responsabilidad exclusiva de interactuar con MySQL mediante comandos parametrizados (MySqlCommand con CommandType.StoredProcedure), liberando a los formularios de cualquier instrucción SQL o referencia a ADO.NET y respetando el Principio de Responsabilidad Única (SRP).

6. Manejo de Errores, Estrategia de Conexión y Decisiones de Diseño

6.1. Estrategia de Conexión Centralizada (Conexion.cs) y App.config

La clase Conexion se declara con el modificador static debido a que representa un servicio utilitario de infraestructura sin estado que no debe ser instanciado como objeto de negocio. Implementa el patrón de factoría simple a través de su método CrearConexion(), el cual retorna una nueva instancia configurada de MySqlConnection pero sin invocar anticipadamente .Open().

Cada método que interactúa con la base de datos envuelve la conexión en un bloque using. Al finalizar el bloque (incluso si se produjera una excepción no controlada), el entorno de ejecución de .NET invoca automáticamente el método Dispose(), cerrando la conexión hacia MySQL y devolviendo el recurso al pool de conexiones del sistema operativo sin riesgo de fugas de memoria o conexiones huérfanas.

Asimismo, se prevé la migración de la cadena de conexión hacia el archivo de configuración externa App.config mediante ConfigurationManager.ConnectionStrings["CoworkingDB"], permitiendo modificar el host, puerto o credenciales de acceso sin requerir la recompilación del código fuente.

6.2. Flujo de Captura del Error 45000 de MySQL a través de MySqlException

Cuando el procedimiento almacenado sp_alta_reserva detecta una violación de disponibilidad mediante fn_verificar_disponibilidad, interrumpe la transacción emitiendo la instrucción SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'El recurso ya se encuentra reservado...'. El proveedor ADO.NET captura dicha señal y arroja una excepción MySqlException hacia la aplicación C#.

```
// Captura y tratamiento controlado de excepciones provenientes de MySQL
try
{
    using (MySqlConnection conn = Conexion.CrearConexion())
    {
        conn.Open();
        using (MySqlCommand cmd = new MySqlCommand("sp_alta_reserva", conn))
        {
            cmd.CommandType = CommandType.StoredProcedure;
            cmd.Parameters.AddWithValue("p_id_cliente", idCliente);
            cmd.Parameters.AddWithValue("p_id_recurso", idRecurso);
            cmd.Parameters.AddWithValue("p_fecha", fecha);
            cmd.Parameters.AddWithValue("p_horas", horas);
```

```
            cmd.Parameters.AddWithValue("p_total", total);
            cmd.Parameters.AddWithValue("p_estado", "Confirmada");

            cmd.ExecuteNonQuery();
            MessageBox.Show("Reserva confirmada con éxito.", "Operación Exitosa",
                            MessageBoxButtons.OK, MessageBoxIcon.Information);
        }
    }
}
catch (MySqlException ex)
{
    // Código 1644 / SQLSTATE 45000 corresponde a errores de negocio emitidos por SIGNAL
    if (ex.Number == 1644 || ex.SqlState == "45000")
    {
        MessageBox.Show("Aviso de Negocio: " + ex.Message,
                        "Conflicto de Disponibilidad", MessageBoxButtons.OK,
MessageBoxIcon.Warning);
    }
    else
    {
        MessageBox.Show("Falla de Base de Datos [Código " + ex.Number + "]: " + ex.Message,
                        "Error Técnico", MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
}
catch (Exception ex)
{
    MessageBox.Show("Error inesperado en el sistema: " + ex.Message,
                    "Error General", MessageBoxButtons.OK, MessageBoxIcon.Error);
}
```

Código 3: Bloque estructurado de captura diferenciada de MySqlException (SIGNAL 45000)

6.3. Manejo General de Excepciones y Ausencia de Bloques Vacíos

En estricto cumplimiento de las directivas institucionales de la cátedra, la totalidad de los bloques catch implementan una respuesta activa (presentando un mensaje explicativo al usuario y preservando la trazabilidad de la pila de llamadas mediante ex.Message). Se garantiza la inexistencia de bloques catch vacíos en todo el proyecto.

6.4. Funcionalidad y Esquema de Intercambio de Datos JSON

Conforme a los requerimientos evolutivos del proyecto, se definió el formato estándar de intercambio de datos mediante JSON (JavaScript Object Notation). Este módulo permite exportar e importar tanto el catálogo de recursos como los lotes históricos de reservas para interoperabilidad con sistemas externos y respaldo de seguridad:

```
[
  {
    "IdReserva": 101,
    "Cliente": {
      "IdCliente": 1,
      "NombreCompleto": "Juan Pérez",
      "Mail": "juan.perez@empresa.com"
    },
```

```
    "Recurso": {
      "IdRecurso": 2,
      "Nombre": "Oficina Ejecutiva A",
      "Tipo": "OficinaPrivada",
      "PrecioPorHora": 4500.00
    },
    "Fecha": "2026-11-15T09:00:00",
    "Horas": 4,
    "TotalCalculado": 19500.00,
    "Estado": "Confirmada"
  }
]
```

Código 4: Estructura del archivo JSON de intercambio para exportación/importación de reservas

6.5. Justificación de Decisiones Técnicas y Alternativas Analizadas

- **Persistencia vía Stored Procedures vs SQL Dinámico:** Se priorizó el uso de procedimientos almacenados sobre sentencias SQL embebidas en el código C# para prevenir vulnerabilidades de Inyección SQL (SQL Injection), reducir el tráfico en la red local y permitir optimizaciones en el plan de ejecución dentro del motor de base de datos.
- **Validación de Disponibilidad en Motor vs Cliente:** Se resolvió delegar la comprobación de disponibilidad a la función fn_verificar_disponibilidad en el motor relacional en lugar de calcularla en memoria del cliente. Esto previene condiciones de carrera (race conditions) en escenarios multiusuario donde dos operadores intentasen reservar simultáneamente el mismo espacio.
- **Mecanismo de Login y Soporte Técnico:** Se implementó una autenticación ágil basada en roles (Usuario Operativo y Administrador). Para el restablecimiento de credenciales, se descartó deliberadamente el envío automático de correos (que requería configurar servidores SMTP propensos a bloqueos) y se optó por un enlace interactivo vía WhatsApp, garantizando asistencia humana inmediata.

7. Manual de Usuario y Guía de Operación del Sistema

7.1. Requisitos Previos para el Encendido del Sistema

- **1. Servidor de Persistencia:** Iniciar el panel de control de XAMPP y activar el servicio de MySQL en el puerto 3306 (debe visualizarse en color verde).
- **2. Inicialización de Datos:** Abrir MySQL Workbench o phpMyAdmin y ejecutar por única vez el script DB/DB_TPFinal.sql para inicializar el esquema tp_final_db con tablas, vistas, funciones, procedimientos y datos de muestra.
- **3. Lanzamiento:** Abrir la solución SistemaDeGestion.slnx en Microsoft Visual Studio 2022 y compilar con F5 (o ejecutar el binario compilado SistemaDeGestion.exe).

7.2. Autenticación de Usuarios (FrmLogin) y Asistencia vía WhatsApp

Al pulsar la opción 'Ingresar como Administrador' desde el menú superior, se despliega el formulario modal FrmLogin. Las credenciales autorizadas de nivel directivo son Usuario: Admin y Contraseña: 123456. Si los datos ingresados no concuerdan, el sistema notifica el campo erróneo y sitúa el foco del cursor para su corrección rápida. En la parte inferior se encuentra el enlace '¿Necesitas ayuda? Contactar soporte', el cual invoca el navegador web o la app de WhatsApp derivando un mensaje tipificado al número de atención oficial (+54 9 223 301-4626).

7.3. Pantalla Principal (FrmPrincipal): Control Operativo y Reservas

- **Solapa CONTROL:** Dispone de selectores vinculados para Cliente (ComboBox), Tipo de Recurso (ComboBox), fecha de inicio (DateTimePicker) y horas de uso (NumericUpDown). El visor de importe calcula el monto total a pagar considerando tarifas base y recargos por equipamiento. El botón 'Agregar' envía la solicitud de reserva al motor.
- **Solapa RESERVAS:** Exhibe una grilla completa (DataGridView) con el listado de reservas vigentes. Permite reprogramar turnos mediante el botón 'Modificar Reserva' o registrar la baja mediante 'Eliminar Reserva', disparando la auditoría automática en MySQL.

7.4. Módulo de Administración: Gestión de Usuarios y Recursos

Una vez validada la clave de administrador, se desbloquea dinámicamente la solapa 'GESTIÓN'. Esta pantalla cuenta con botones de opción mutuamente excluyentes (RadioButtons: 'Usuarios' y 'Recursos') que conmutan instantáneamente la grilla activa entre el catálogo de Clientes y el catálogo de Espacios Físicos. Desde allí, los botones 'Crear', 'Modificar' y 'Eliminar' permiten ejecutar el mantenimiento integral del padrón. La opción

'Cerrar sesión' en el menú superior oculta la solapa administrativa para resguardar la seguridad de la información.

8. Guía de Ejecución y Puesta en Marcha

Para reproducir el entorno y ejecutar las pruebas de validación académica en cualquier equipo de prueba, siga los pasos estandarizados:

- **Paso 1 (Descarga de Fuentes):** Clonar o descomprimir el repositorio del proyecto en una ruta local accesible (ej. c:\xampp\htdocs\TP_FINAL_ALGORITMOS).
- **Paso 2 (Despliegue de Base de Datos):** Abrir XAMPP Control Panel, inicializar el servicio MySQL en el puerto 3306, ingresar a phpMyAdmin (http://localhost/phpmyadmin) o MySQL Workbench, abrir el archivo DB/DB_TPFinal.sql y hacer clic en 'Ejecutar'. Verificar la creación de la base tp_final_db y sus 5 tablas.
- **Paso 3 (Apertura de Solución):** Abrir Visual Studio 2022 y cargar el archivo de solución SistemaDeGestion.slnx ubicado en la carpeta del proyecto.
- **Paso 4 (Restauración de Dependencias):** Hacer clic derecho sobre la solución en el Explorador de Soluciones y seleccionar 'Restaurar paquetes NuGet' para garantizar que MySql.Data se encuentre debidamente vinculado.
- **Paso 5 (Compilación y Ejecución):** Seleccionar la configuración 'Debug' / 'AnyCPU' y presionar la tecla F5 (o el botón Iniciar). La aplicación compilará sin errores y desplegará la pantalla de inicio.

9. Referencias Bibliográficas (Normas APA 7.ª edición)

A continuación se listan las fuentes técnicas y bibliográficas formales consultadas para el desarrollo del software y la redacción de la presente memoria técnica, conforme a las pautas de citación APA 7.ª edición:

Freeman, E., & Robson, E. (2020). *Head First Design Patterns: Building Extensible and Maintainable Object-Oriented Software* (2.ª ed.). O'Reilly Media.

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional.

Martin, R. C. (2018). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.

Microsoft Corporation. (2023). *Documentación de C# y Guía de Arquitectura de .NET Framework*. Microsoft Learn. https://learn.microsoft.com/es-es/dotnet/csharp/

Oracle Corporation. (2024). *MySQL 8.0 Reference Manual: Stored Procedures, Triggers and Functions*. Oracle Help Center. https://dev.mysql.com/doc/refman/8.0/en/

Troelsen, A., & Japikse, P. (2022). *Pro C# 10 with .NET 6: Foundational Principles and Practices in Programming* (11.ª ed.). Apress.

10. Checklist Institucional de Control Previo a la Entrega

Tabla de control de conformidad previa respecto a las Pautas de Presentación de la Cátedra:

Requisito Institucional	Criterio de Evaluación	Estado
Portada Reglamentaria	Instituto, carrera, materia, docente, año, versión y datos completos de todos los integrantes.	CUMPLE (100%)
Formato de Documento	Hoja tamaño A4, márgenes de 2,54 cm en los cuatro costados, texto justificado.	CUMPLE (100%)
Tipografía y Estilo	Arial 11 pt, interlineado 1,5, jerarquía de títulos estricta (18 pt, 16 pt, 13 pt, 11 pt).	CUMPLE (100%)
Normas APA 7.ª Edición	Rotulado formal de figuras, tablas sin bordes verticales y sección final de referencias.	CUMPLE (100%)
Índice y Paginación	Tabla de contenidos con puntos guía y numeración de página arábiga en pie de página.	CUMPLE (100%)
Modelo de Base de Datos	DER exportado de Workbench, script DB_TPFinal.sql, tablas, vistas, función y trigger.	CUMPLE (100%)
Validación SIGNAL 45000	Control estricto de disponibilidad y captura diferenciada vía MySqlException en C#.	CUMPLE (100%)
Arquitectura POO y C#	Clase abstracta Recurso, interfaz IReservable, herencia y polimorfismo en CalcularCosto.	CUMPLE (100%)
Gestión y Capas (DAO)	Uso de GestoraCoworking con LINQ y desacoplamiento progresivo mediante GestorDAO.	CUMPLE (100%)
Código y Evidencias	Bloques de código con tipografía monospace sin capturas innecesarias de texto.	CUMPLE (100%)